Interfaces in Java :

Using interface you can specify what a class must do, but not how it does.

Interfaces are similar to class but they donot contain instance variable and methods are declared without body i.e methods are abstract.

Once a interface is designed any number of class can impliment an interface.

One class can impliment any number of interfaces. To impliment an interface a class must provide method definition required by interfaces.

Defining Interface :

```
access  interface-name {
        return-type method-name1(parameter-list);
        return-type method-name2(parameter-list);
        type final-variablename1 = value;
        type final-variable-name2 = value;
        :
        return-type methodnameN (parameter-list);
        type final-varnameN = value;
}
```

Variables inside interface are implicitly final and static, that means they cannot be changed.

Variables must be initialized. All methods and variables are implicitly public.

Example:

```
// Ifs1.java
interface Ifs1
{
    int a = 10;
    int b = 20;
    void f1();
    void f2();
}
```

Compile the above program, then write javap Ifs1 ↵

output will be:

```
interface Ifs1
{
    public static final int a;
    public static final int b;
    public abstract void f1();
    public abstract void f2();
}
```

Inheriting the features of interfaces:

In order to inherite the features of interfaces to derived class or derived interfaces we have three approaches.

Approach ① Inheriting features of interface to derived class.

```
[abstract] class <class-name> implements
        <interface1> <interface2> ... <interfaceN>
{
    variable declaration;
    method definition;
}
```

implements is a keyword used for inheriting the features of interfaces to the derived class.

If a derived class is not defining at least one abstract method then the definition of the derived class must be made as abstract.

Example:

```
interface I1
{
    void f1();
}

interface I2
{
    void f2();
}

class c1 implements I1, I2
{
    public void f1() {
        sop("f1 defined in c1");
    }
    public void f2() {
        sop("f2 defined in c1");
    }
    void f3() {
        sop("f3 method of c1");
    }
}

class InterfaceTest
{
    PSVM(-) {
        C1 o1 = new C1();
        o1.f1();
        o1.f2();
        o1.f3();

        I1 oi1 = new c1();
        oi1.f1();
        I2 oi2 = new c1();
        oi2.f2();
        // oi2.f3();
    }
}
```

Points to remember :

1. Interfaces are by default abstract.
2. Interfaces doesnot contain constructors.
3. Interfaces should not contain main() method.
4. Interfaces doesnot contain information about additional method defined in derived class.
5. While we are defining the methods of interface in the derived class the methods must be preceded by public.
6. Reference to an interface can be created and it can hold an object created by its derived class.

Approach-2 : (Interface Inheritance)

The process of getting features of interface to the derived ~~class~~ interface is known as interface inheritance.

Syntax :

```
interface <interface name> extends <interface-name1>,
                <interface-name2>, ... <interface-namen>
{
    variable declaration;
    method declaration;
}
```

In java programming one class can extends only one class but one derived interface can extends either one or more interfaces.

Example

```
interface i1 {
        void f1(); }
interface i2 extends i1
        void f2(); }

class c1 implements i2
{       public void f1() {
            sop(" f1() of c1");
            }
        public void f2() {
            sop("f2() of c1");
          }
        void f3() {
            sop("f3() special");
}

class IntfcDemo
{       PSVM(-)
        {   c1 o1 = new c1();
            o1.f1();
            o1.f2();
            o1.f3();
        }
}
```

output

```
f1() of c1
f2() of c1
f3() - special
```

Approach-3:

This approach inherites features from both class and interfaces into the desired class.

Syntax:

[abstract] class <class-name> extends <class-name>
implements <interface1> <interface2> ... <interfaceN>

{
    variable declaration;
    method declaration/definition;
}

Important:

Where we use both "extends" and "implements" keyword in a single syntax, it is mandatory for the programmer to use extends first and then implements.

Example:

```
interface I1
{   void f1();
}
interface I2 extends I1
{ void f2();
}
abstract class C2 extends C1 implements I2
{     public void f1(){
          sop("f1() of C2-original");
      }
      void f3(){
          sop("f3() of C2-overridden");
      }
}
```

```
abstract class C1
{     void f3()
      {}
}
```

```
class C3 extends C2
{
    public void f1() {
        sop("f1() of C3 - overridden");
    }
    public void f2() {
        sop("f2() of C3 - original");
    }
    public void f3() {
        sop("f3() of C3 - 2nd overridden");
    }
    void f4()
    {   sop("f4() of C3 - Special");
    }
}

class IntfDemo2
{   psvm(-) {
        C3 o3 = new C3();
        o3.f1();
        o3.f2();
        o3.f3();
        o3.f4();
    }
}
```

output

```
f1() of C3 - overridden
f2() of C3 - original
f3() of C3 - 2nd. overridden
f4() of C3 - Special.
```

Packages in JAVA

Package in Java is a mechanism to encapsulate a group of classes, sub packages and interfaces.

Packages are used for following reasons:

(1) Preventing naming conflicts. For example there can be two classes in two packages like college.math.Employee and college.cs.Employee.

(2) making search and usage of classes, interfaces, enumerations and annotations easier.

(3) Providing controlled access to java class, variable and methods.

(4) Packages can be used for data hiding.

Defining a package:

To create a package, include a package command as the first statement in a java source file.

Any classes declared within that file will belong to the specified package.

When you are not defining your package then the class will be put into default package with no name.

Example: package MyPackage;

Java uses file system directories to store packages. i.e. .class files of any class you declare to be part of MyPackage must be stored in the directory MyPackage.

more than one file can include same package statement. All the classes created by these files will be stored inside that package.

You can create hierarchy of packages. The general form is :

```
package pkg.[pkg2.[pkg3]];
```

Example : package pkg.new.examples;

Finding packages and CLASSPATH :

How does the Java run-time system knows where to look for packages that you created ?

Answer : (1) First by default, the JAVA run-time system uses the current working directory as starting point.

(2) second you can specify a directory path or paths by setting the CLASSPATH environment variable.

(3) You can use the -classpath option with java and javac to specify path to your classes.

Example : Simple.java

```
package pack.newexamples;
class Simple
{ PSVM(-) {
    System.out.println("Hello Package");
  }
}
```

Compile : javac -d . Simple.java ↵

(create directory) (current directory)

Run : java pack.new.examples.Simple ↵

Output

Hello package

Access Protection :

classes and packages are both used for encapsulating and containing namespace scope for variables and methods.

Packages act as containers of classes, interfaces and subpackages. classes act as containers for data and methods.

JAVA uses four categories of visibility for class members.

- Subclass in the same package.
- Non-subclass in the same package.
- Subclass in different package.
- classes that are neither same package nor subclasses.

class member access matrix :

	Private	No modifier	Protected	Public
Same class :	Yes	Yes	Yes	Yes
Same package subclass :	NO	Yes	Yes	Yes
Same package non-subclass :	NO	Yes	Yes	Yes
Different package subclass :	NO	NO	Yes	Yes
Different package non-subclass :	NO	NO	NO	Yes.

Anything declared public can be accessed from anywhere.

Anything declared private cannot be seen outside the class.

When no access specifier i.e default then the member is visible to subclass as well as other class inside the package

If you want to allow member to be accessed from outside package but only to class that is subclass then declare the element protected.

Example :

```
// Protection.java
package p1;
public class Protection {
    int n = 1;
    private int n_pri = 2;
    protected int n_pro = 3;
    public int n_pub = 4;
    public Protection() {
        sop("base constructor");
        sop("n = " + n);
        sop("n_pri = " + n_pri);
        sop("n_pro = " + n_pro);
        sop("n_pub = " + n_pub);
    }
}
```

```
// Derived.java
package p1;
class Derived extends Protection {
    Derived() {
        sop("derived constructor");
        sop("n = " + n);
        // sop("n_pri = " + n_pri);
        sop("n_pro = " + n_pro);
        sop("n_pub = " + n_pub);
    }
}
```

```
// SamePackage.java
package p1;
class SamePackage {
    SamePackage() {
        Protection p = new Protection();
        sop("same package constructor");
        sop("n = " + p.n);
        // sop("n_pri=" + p.n_pri);
        sop("n_pro =" + p.n_pro);
        sop("n_pub=" + p.n_pub);
    }
}
```

```
// Protection2.java
package p2;
class Protection2 extends p1.Protection {
    Protection2() { sop("derived other package constructor"); }
    // sop("n = " + n);
    // sop("n_pri = " + n_pri);
    sop("n_pro =" + n_pro);
    sop("n_pub =" + n_pub);
}
```

```
// OtherPackage.java
Package p2;
class OtherPackage {
    OtherPackage() {
        p1.Protection p = new p1.Protection();
        sop("other package constructor");
        // sop("n=" + p.n);
        // sop("n_pri=" + p.n_pri);
        // sop("n_pro=" + p.n_pro);
        sop("n_pub=" + p.n_pub);
    }
}
```

(30)

```
// Demo package P1.
package p1;
public static Demo {
    PSVM(-) {
        Protection ob1 = new Protection();
        Derived ob2 = new Derived();
        SamePackage ob3 = new SamePackage();
    }
}

// Demo package P2.
package P2;
public class Demo {
    PSVM(-) {
        Protection2 ob1 = new Protection2();
        OtherPackage ob2 = new OtherPackage();
    }
}
```

Importing packages :

Java uses import statement to bring certain class or entire package into a program.

Example :

```
import java.util.Date;  // single class import
import java.io.*;       // import entire package.
```

import statement is optional. Any place you can use a class of an existing package by fully qualified name (which includes full package hierarchy).

Example :

```
import java.util.*;
class MyDate extends Date {
}
```

Without import it can be written as :

class MyDate extends java.util.Date {

}

So there are three ways to access the package from outside packages.

(1) import package.*;
(2) import package.classname;
(3) fully qualified class name.

Some Predefined Packages in JAVA:

(1) java.lang.*;

This package is used for achieving language specific functionalities. Some of them are:

Use of Garbage Collector.
Converting the data from String to fundamental and from fundamental type to String. (wrapper class)
Displaying result of java program in console.

*** This package is by default imported to every Java program.

(2) java.io.*;

This package contains collection of classes and interfaces which are used for performing various IO operations or file operation.

Some of these operations includes
- Opening a file either read or write mode.
- Reading a record from file
- Insert a record into the file
- Modifying or deleting a record from the file.

③ java.net.*;

This package is used for developing client-server application and networking application.

In client-server application development we used to write two types of programs:
- Client program
- Server program

For writing client program we need to use the set of classes and interfaces present in java.net.*;

Similarly for writing server side program we require set of classes and interfaces of java.net.*; package.

④ java.util.*; (Collection framework)

It contains the collection framework, legacy collection classes, event models, date and time facilities and miscellaneous classes (a string tokenizer, a random-number generator and a bit array).

—— o ——